Data exploration

Team Unsupervised Predictors

Oleksandr Kudriavchenko **Volodymyr Halian**

1 Verifying Annotations & Qualitative Analysis

1.1 Overall expression

Our team has verified 10 annotations and results are pretty interesting. Those annotations usually had disagreements:

- small shifts in on-/offsets times

- choose wrong class labels

- missing class labels

- some non-target sounds were classified as target ones

- splitting/merging soundevents of the same class

Does it mean that those differences imply violation of the guidelines by one of the annotator? - Not always. we've noticed few examples, where same target events were separated by small time breaks. According to the guidelines,for most of the classes we can merge those together if the break is smaller than one second. But in cases that are close to one second, merging into one or splitting into several events can both be correct solutions.

1.2 Rule violation

Still, we've noticed true violations. One of those

Filename: 003591.wav

In this example, a clear violation is presented. In the Figure 1 we can see that one of the annotators clearly missed one target class - phone_ringing (Fig. 2), while the scene description says: "...I then opened the door and walked into the room while my tablet was ringing...". This disagreement is likely occurred because the phone ringing sound partially overlaps with other target and background sounds, which may mask the phone ringing signal, so make it less precise. However, if the annotator had carefully reviewed the scene description, he might have noticed that there was a mention of phone_ringing.

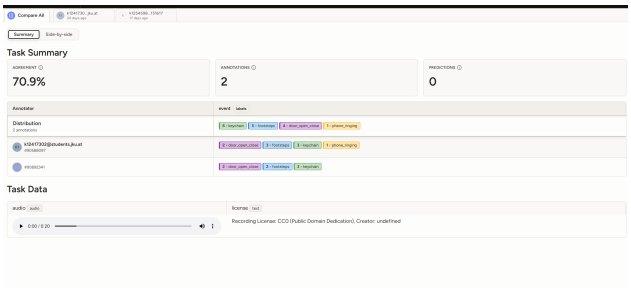


Figure 1: Labeling summary

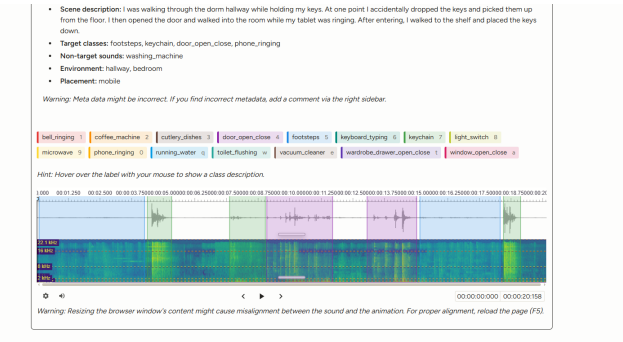


Figure 2: Annotation with an example of violation

2 Annotations

2.1 Annotator Agreement

Annotator Agreement was computed using time-aligned annotation arrays binarized with a threshold of 0.5. Consistency was measured using naive agreement and Cohen's kappa, which accounts for agreement by chance. As shown in Table 1, the average naive agreement was 98.36%, while Cohen's kappa was 0.8258. The difference between these values is caused by class imbalance, since most time steps correspond to background. Class-wise results (Table 2) indicate higher agreement for distinct sounds such as *bell_ringing* and lower agreement for more ambiguous events like *footsteps*.

2.2 Convert Annotations to Labels

Final binary labels were derived using majority voting across annotators. Labels were averaged and assigned when the mean was at least 0.5. Comparing the derived labels with the metadata, 72.68% of recordings showed an exact match of target tags, while 81.13% contained all target classes. This indicates that majority voting generally recovers the intended labels, but short events may still be missed.

2.3 Label Characteristics and Dataset Distribution

We analyzed class frequency and average event duration using the majority-vote labels. As shown in Figure 3, the dataset is imbalanced, with footsteps, door_open_close, and cutlery_dishes occurring most frequently, while coffee_machine and bell_ringing appear less often.

Figure 4 shows that event duration varies across classes. Continuous sounds such as vacuum_cleaner and running_water have longer durations, whereas short sounds such as light_switch and door_open_close are significantly shorter. These results indicate both class imbalance and temporal variability in the derived labels.

Table 1: Annotator Agreement and Label Conversion Results

Category	Metric	Value	Interpretation
Annotator Agreement	Naive Agreement	98.36%	High agreement dominated by non-event regions due to class imbalance.
Annotator Agreement	Cohen's Kappa	0.8258	Strong agreement after correcting for chance-level matching between annotators.
Label Conversion	Exact Target Tag Match	72.68%	Derived labels exactly match the original target class tags for most recordings.
Label Conversion	All Target Tags Found	81.13%	Majority voting recovers all expected target classes in the majority of recordings.

Table 2: Class-wise Annotator Agreement

Class	Agreement	Class	Agreement
bell_ringing	0.9936	light_switch	0.9924
coffee_machine	0.9925	microwave	0.9834
cutlery_dishes	0.9738	phone_ringing	0.9822
door_open_close	0.9678	footsteps	0.9276
keyboard_typing	0.9817	keychain	0.9772
running_water	0.9792	toilet_flushing	0.9860
vacuum_cleaner	0.9919	wardrobe_drawer_open_close	0.9832
window_open_close	0.9899		

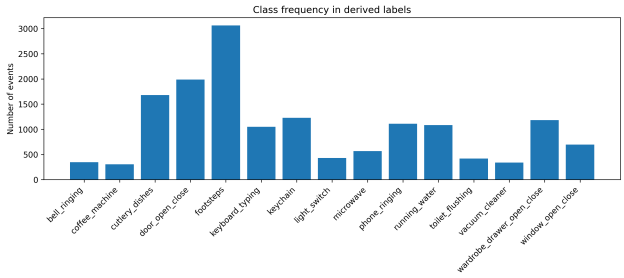


Figure 3: Frequency of derived sound events for each class.

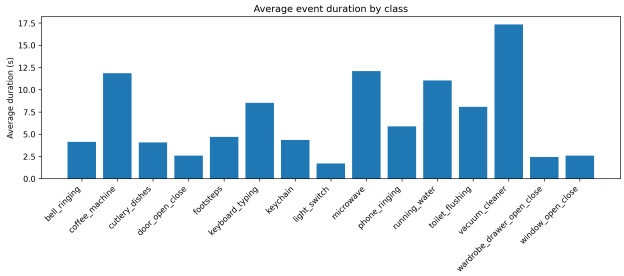


Figure 4: Average duration of derived sound events by class.

3 Audio features

Now we want to analyze the audio features. To do this we'll firstly look at metadata. The results of our work is displayed in graphs in Figure 5 and explained in following subsections.

The distribution of the recording devices graph shows us the devices that user used for recording the audio files. As we can see more that a half of devices are iPhones, next big part belongs to Samsung devices. Next categories are Google and Xiaomi devices. Other device manufacturers grouped together into "other"category.

In **Device placement distribution** graph we want to show that majority of the recordings are done in static.

Top recording locations graph shows most common recording environments. Huge part of those recordings are made in kitchens, then there go bedrooms, hallways, living rooms etc. It leads us to conclusion that kitchen is the place, where it was easier to do the recordings for most of the target classes. And the graph itself can be used to prioritize microphone placement in smart home systems.

Number of target classes per recording graph we can see that most of the recordings have 3 or 2 target classes. Third place take recordings with 4 classes. Then to explain two last bars we need to remember recording guidelines. There are some recordings that consist only of one target class, because it was said the 6 out of 8 recordings must contain at least two classes, that is the reason, why we don't have much of those in dataset. And also, there was a limit of 5 classes per scene, and in this case it most probably be a high-polyphonic scene, which was a bonus task.

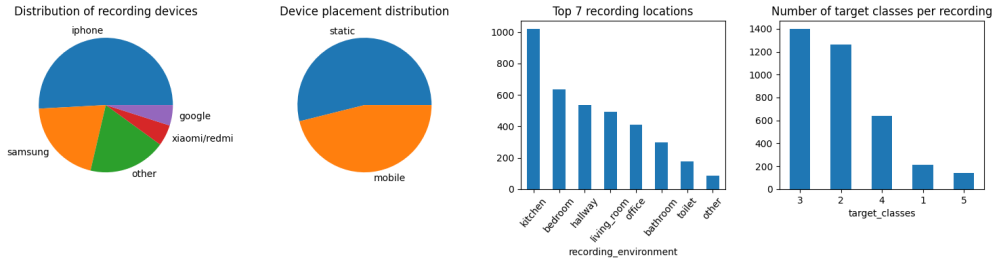


Figure 5: Features from metadata

3.1 Feature statistics

Then we've calculated distributions of features for audio files. In the dataset the feature information was aggregated with time windows. For analysis we've aggregated mean, std, min and max firstly for each file, getting average std value, average mean value and global min/max values. In the same way values were aggregated for a whole dataset for 9 features to get summary statistics.

In the Figure 6 demonstrated 9 boxplot-like graphs, but they are modified to represent the values we got. Whiskers represent min and max values, while orange line in a box represent the mean value. Box limits represent mean$\pm$std .

ZCR is a count of zero crossings. With it we can notice, which frequencies dominate in recording. In our dataset we can see that most values are rather low, so lower frequencies dominate in recordings.

Flux shows how much spectrum changes over time by measuring distance between neighboring frames in it. Graph shows us that most of the values are low, so spectrums are stable, for example, we may have a recording with running_water class through the most of the timeline.

Flatness shows how noisy is the signal. 1 is just noise, and 0 is a clear tone, like a cosine wave. The information from the graph says that the value of noise is not high. We assume, the biggest class of the recording devices was an iPhone, which have quality microphones influenced this metric.

Centroid shows where is "the center of the mass" in the spectrum (a weighted average of frequencies). **Bandwidth** measures how the spectrogram spreads around the centroid. In the dataset the mean centroid lies around 2000 Hz, and most of the standard derivation values also point out the range between 1kHz and 3kHz. Mean bandwidth is located around 2,8kHz. Theoretically we are covering range between 0Hz and 6kHz. It makes sense considering the target classes in the recordings and can be a way of optimization.

Rolloff low and high are approximations of the lowest and highest frequencies of the spectrum. Graphs supports the conclusion from previous metrics, and also tell us about a range between 0Hz and 6kHz.

Energy explains how loud is sound, while **power** shows how unsmooth is the signal. Noise gets reduced while sound spikes are gettingstronger and noticeable on a graph. From our graphs we see that most of the recordings share pretty similar volume levels and smoothness but there are some cases, when recordings are louder, or loud sound spikes appear there. It also says that we should normalize our recordings before training the classifier, so it has same reference points.

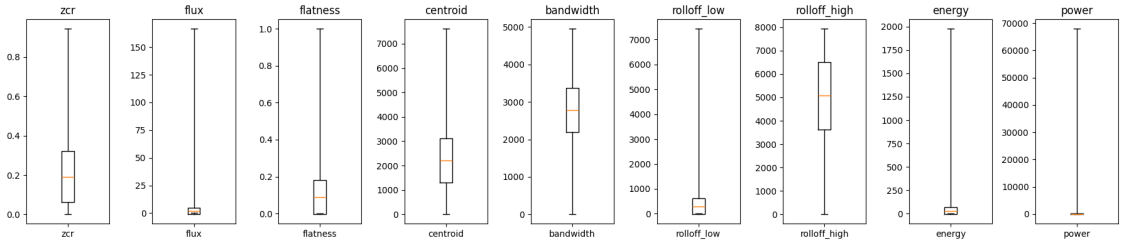


Figure 6: Enter Caption

3.2 Feature correlation

To analyze the correlation, we've calculated the correlation matrix and created a heatmap (Figure 7). We see that **energy** and **power** are correlated with **flux**. It makes sense, because when flux is on average higher for some record, it means

that there are rapid changes in spectrogram. The power also gets higher then we've spikes of volume, and when there is a lot of those spikes throughout the recording, then the average volume is higher too, which is showedwith energy. Also we've noticed that there is correlation between **ZCR** and **centroid**. The last one represents "most important" frequency of the recording and the other - zero crossing rate, which can be interpreted in similar way.

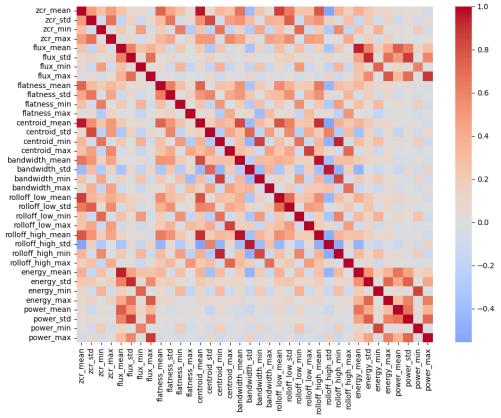


Figure 7: Correlation matrix

3.3 Feature space visualization

We merged table with previously described features (file-wise features) with metadata frame. So now we have audio features together with target class labels. Two classes that came to our mind were running_water and toilet_flushing, so we wanted to see, if those data points will be close together. The plot (Figure 8) was created, where each point is a recording. To reduce dimensionality we used T-SNE algorithm with perplexity = 30 and n_components = 2. We colored target classes with two shades of green and others with gray. From the plot we see that green points frequently concentrate together, what implies that people usually follow hygiene rules.

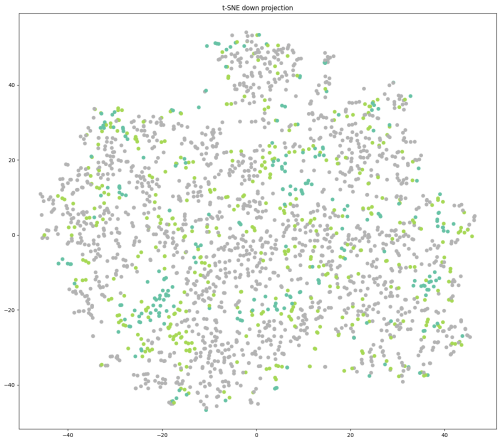


Figure 8: toilet_flushing and running_water

4 Conclusion

4.1 Biases

Firstly, class imbalance is present, as some sounds such as footsteps and door_open_close occur much more frequently than others like bell_ringing or coffee_machine. This may lead the model to favor frequent classes and perform poorly on rare events.

Secondly, some sounds have unclear onset and offset boundaries. For example, footsteps are harder to annotate consistently, causing different temporal annotations and lower agreement between annotators.

Thirdly, recording environment bias exists because most sounds were recorded in similar domestic environments. This may limit generalization to unseen acoustic conditions or background noises placements.

4.2 Dataset Suitability for Classification

The dataset is suitable for training sound event detection models, as it provides different classes with time-aligned annotations and majority-vote labels. Overall annotator agreement is relatively high, indicating consistent annotations. However, learning a reliable classifier remains challenging due to class imbalance, ambiguous onset and offset boundaries for some sounds and large differences in event duration. These factors may introduce noisy labels and bias the model. Despite these drawbacks, the dataset is appropriate for the next phase, but techniques such as class balancing and temporal smoothing may be required to improve performance.

4.3 Broader Impact

The dataset can enable useful applications such as smart home automation, safety detection and assistive technologies. Systems trained on this data could recognize some background sounds or unusual activities. However, audio monitoring may raise privacy concerns if used for surveillance in private environments. To minimize these risks, the dataset should exclude sensitive recordings, anonymize metadata, and include clear ethical usage guidelines.

5 Disclosure of LLM and AI Tool Use

LLM's were used while preparing this report for fast access to python code examples (like graph generation), broader understanding of the concepts or grammar correction. Verification was done logically in case of code snippets, or checked in with lecture slides in case of understanding the meaning of new terms.Additionally, we used LLM to generalize and summarize some parts of the analysis to avoid unnecessary verbosity.